

中国科学院大学

《计算机组成原理(研讨课)》实验报告

姓名 吴竞宸 学号 2024K8009929022 专业 计算机科学与技术
实验项目编号 4.3 实验名称 深度学习算法与硬件加速器

- 注 1: 撰写此 Word 格式实验报告后以 PDF 格式保存 SERVE CloudIDE 的 `/home/serve-ide/cod-lab/reports` 目录下(注意: reports 全部小写)。文件命名规则: `prjN.pdf`, 其中 `prj` 和后缀名 `pdf` 为小写, `N` 为 1 至 4 的阿拉伯数字。例如: `prj1.pdf`。PDF 文件大小应控制在 5MB 以内。此外, 实验项目 5 包含多个选做内容, 每个选做实验应提交各自的实验报告文件, 文件命名规则: `prj5-projectname.pdf`, 其中“-”为英文标点符号的短横线。文件命名举例: `prj5-dma.pdf`。具体要求详见实验项目 5 讲义。
- 注 2: 使用 `git add` 及 `git commit` 命令将实验报告 PDF 文件添加到本地仓库 master 分支, 并通过 `git push` 推送到实验课 SERVE GitLab 远程仓库 master 分支(具体命令详见实验报告)。
- 注 3: 实验报告模板下列条目仅供参考, 可包含但不限定如下内容。实验报告中无需重复描述讲义中的实验流程。

一、设计成果

1 卷积与池化算法实现

本实验在 RISC-V 处理器上实现了 CNN 推理中的标准卷积和最大池化(max pooling)操作, 输入为 28×28 的单通道特征图, 使用 20 个 5×5 卷积核进行卷积, 然后对 20 个输出通道分别进行 2×2 最大池化。

卷积操作采用五层循环结构: 输出通道(oc)、输出高度(oh)、输出宽度(ow)、卷积核高度(kh)、卷积核宽度(kw)。每个输出位置先加载对应通道的偏置值(bias)并左移 `FRAC_BIT` 位转为 32 位定点数作为初始累加值, 然后在 5×5 滑动窗口内对输入特征图与卷积核权重进行乘累加(MAC)运算。乘累加通过自定义的 `mul()` 函数完成, 该函数在硬件乘法指令可用时直接调用 `MUL` 指令(`a * b`), 否则回退到软件乘法(`mul_ll`)。累加完成后结果右移 `FRAC_BIT` 位恢复 16 位定点格式存储。

池化操作同样采用五层循环结构, 对每个输出通道的每个 2×2 窗口取最大值。与卷积共用同一块存储区, 直接读取卷积输出数据进行原位处理。

2 CPU 乘法指令扩展

为加速 DNN 推理中的大量乘法操作, CPU 增加了对 `MUL` 指令(RISC-V M 扩展)的支持。在 EX 级通过 `ID_is_mul` 信号识别乘法指令, 结果由组合逻辑乘法(`src1 * src2`)产生, 作为 `EX_result` 的一个来源参与流水线数据通路。

在软件层面, 通过条件编译宏 `USE_MUL` 来选择乘法实现路径: 定义 `USE_MUL` 时使用硬件乘法指令(`a * b`), 未定义时回退到软件乘法函数 `mul_ll`(使用移位加算法实现)。

3 硬件加速器接口

硬件加速器通过 MMIO(内存映射 I/O)方式挂载在地址空间 `0x60030000` 处, 包含两个 32 位寄存器: `START` 寄存器(偏移 `0x00`)和 `DONE` 寄存器(偏移 `0x08`)。软件通过写入 `START` 为 1 启动加速器, 然后轮询 `DONE` 的第 0 位等待加速器完成。硬件加速器完成后由软件读取结果并与参考数据比对验证。

加速器内部可能使用了更大的并行度和流水线深度、专用数据通路来加速卷积和池化计算, 其具体架构不在本实验软件部分讨论范围内。

使用条件编译宏 `USE_HW_ACCEL` 控制是否使用硬件加速器: 定义时调用 `launch_hw_accel()`, 未定义时软件执行 `convolution()` 和 `pooling()`。

二、开发成果

1 卷积核心代码

```
for (oc = 0; oc < weight_size.d0; oc++) {
    for (oh = 0; oh < conv_out_h; oh++) {
        for (ow = 0; ow < conv_out_w; ow++) {
            int sum = ((int)weight[oc * kernel_len]) << FRAC_BIT;
            for (kh = 0; kh < weight_size.d2; kh++) {
                for (kw = 0; kw < weight_size.d3; kw++) {
                    int ih = oh * stride - pad + kh;
                    int iw = ow * stride - pad + kw;
                    if (ih >= 0 && ih < input_fm_h
                        && iw >= 0 && iw < input_fm_w) {
                        short in_val = in[ih * input_fm_w + iw];
                        short w_val = weight[oc * kernel_len + 1
                                                + kh * weight_size.d3 + kw];
                        sum += mul(in_val, w_val);
                    }
                }
            }
            out[oc * conv_out_h * conv_out_w + oh * conv_out_w + ow]
                = (short)(sum >> FRAC_BIT);
        }
    }
}
```

卷积输出尺寸的计算公式为: $\text{output} = (\text{input} - \text{kernel} + 2 * \text{pad}) / \text{stride} + 1$ 。本实验中 $\text{input}=28$, $\text{kernel}=5$, $\text{pad}=0$, $\text{stride}=1$, 因此卷积输出为 24×24 。经过 2×2 池化 ($\text{stride}=2$) 后最终输出为 $12 \times 12 \times 20$ 的三维张量。 $\text{kernel_len} = 1(\text{bias}) + 25(\text{weight}) = 26$ 个 short。

2 乘法选择与硬件加速器启动

```
int mul(short a, short b) {
#ifdef USE_MUL
    int ans = mul_ll(a, b);
#else
    int ans = a * b;
#endif
    return ans;
}

#ifdef USE_HW_ACCEL
void launch_hw_accel() {
    volatile int* gpio_start = (void*)(GPIO_START_ADDR);
```

```
volatile int* gpio_done = (void*)(GPIO_DONE_ADDR);
*gpio_start = 1;
while ((*gpio_done & 1) == 0);
*gpio_start = 0;
}
#endif
```

3 性能测量与结果验证

使用 RISC-V 的 mcycle CSR 寄存器精确测量执行周期数。验证过程将输出结果与预置的参考数据逐字节比对,比对通过后触发 good_trap。

三、性能分析

本实验存在三个可组合的对比维度:软件乘法 vs 硬件乘法(USE_MUL),纯软件 vs 硬件加速器(USE_HW_ACCEL)。通过不同的条件编译组合,可以得到以下四种实现方案:

方案	乘法方式	加速器	说明
A	软件乘法	否	纯软件实现,基线
B	硬件 MUL	否	CPU 加速乘法
C	软件乘法	是	加速器执行,软件乘法仅用于比对
D	硬件 MUL	是	完整硬件加速

表 1: 四种实现方案

方案 A 到方案 B 的加速比直接反映了 RISC-V M 扩展中单周期乘法指令对 DNN 工作负载的提升效果。卷积操作中包含大量乘累加运算(本实验约 $20 \times 24 \times 24 \times 25 = 288,000$ 次乘法),使用硬件乘法指令消除软件乘法的数十周期开销后,预期获得显著加速。

方案 A 到方案 C 的加速比体现了专用硬件加速器相对于通用 CPU 软件实现的性能优势。硬件加速器可通过更宽的并行数据通路、流水线化的 MAC 阵列等架构级优化在数个或数十个周期内完成整个卷积 + 池化操作,而软件实现在 CPU 上需要数百万周期。

四、学习总结

1 收获总结

类别	名称	体会
亮点	条件编译设计	多方案对比只需切换宏定义
亮点	性能测量	CSR 寄存器提供精确周期计数
不足	定点数调试	移位位数与数据精度的平衡需反复试验

表 2: 收获总结

五、扩展思考

- 四种实现方案的性能差异分析。

方案 A (纯软件)效率最低。软件乘法(mul_ll)采用移位加算法,每次乘法需数十个周期,而卷积操作包含约 28.8 万次乘法,乘法开销主导了总执行时间。

方案 B (硬件 MUL)利用 CPU 新增的单周期乘法指令替代软件乘法,消除了每次乘法的数十周期开销。但整体执行仍受限于 CPU 的标量执行和循环控制开销(load/store、地址计算、分支等)。

方案 C (硬件加速器)将整个卷积 + 池化操作卸载到专用硬件, CPU 仅需写入 START 寄存器后轮询 DONE。加速器内的并行 MAC 阵列可在一个或数个周期内完成整个层的计算。瓶颈转移到加速器内部的数据通路宽度和内存带宽。

方案 D (硬件 MUL+ 加速器)在方案 C 基础上, CPU 端的验证比对循环也受益于硬件乘法指令,进一步减少验证阶段的周期数。

● 定点数精度对 DNN 推理的影响。

本实验采用 Q10.6 定点格式(FRAC_BIT=10)。卷积累加过程中中间结果以 32 位存储,有效避免了 16 位累加时的溢出问题。最终右移 10 位截断回 16 位输出,存在舍入误差。对于本实验的小规模推理任务而言,Q10.6 格式已能保证结果的正确性,验证比对全部通过。在更大规模的 DNN 推理中,可能需要考虑更高精度的定点格式或块浮点(block floating point)方案来平衡精度与硬件开销。

六、 实验时长

在课后,你花费了大约 2 小时完成此次实验,其中各个阶段的用时如表 3所示。

步骤	时长(小时)	说明
设计	1	算法逻辑与定点数方案设计
开发	1	卷积/池化 C 代码、mul 指令集成、加速器接口

表 3: 实验时长